



COMP3011

计算机体系结构

3.1 AI处理器体系结构

授课教师：王强

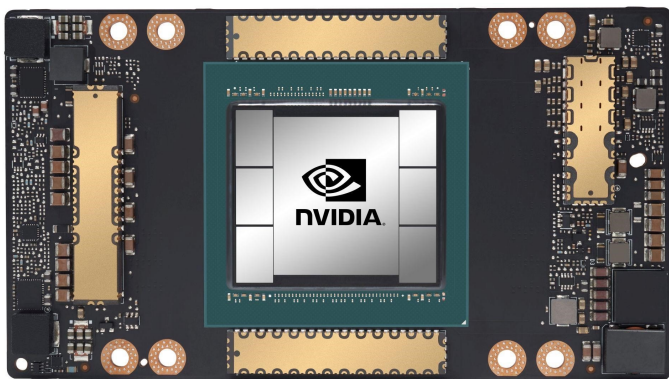
邮箱：qiang.wang@hit.edu.cn



目录 | CONTENT

- 1 人工智能芯片发展介绍
- 2 数据级并行

人工智能芯片发展介绍

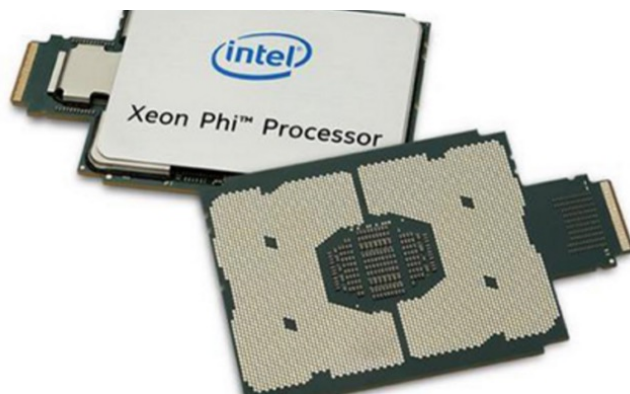


GPU: Graphics Processing Unit

NVIDIA GeForce 8800 GPU

Released at: 2006

Along with CUDA

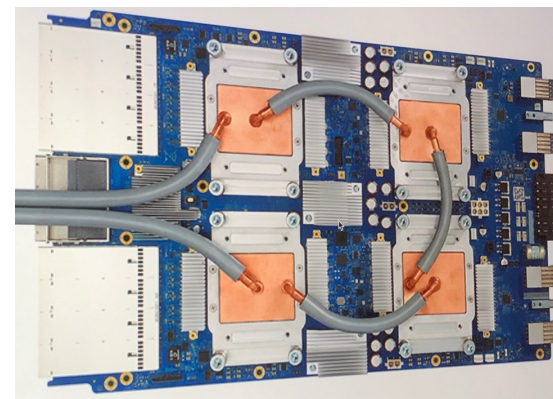


CPU: Central Processing Unit

Intel Xeon Phi processors

Released at: 2010

Knights Landing (KNL) architecture



TPU: Tensor Processing Unit

Google's Tensor Processing Unit (TPU)

Released at: 2016

TPUv1 or TPU Accelerator

以及其他ASIC加速芯片，如寒武纪（中科院）、昇腾（华为）、IPU（Graphcore）、FPGA相关加速器



性能度量

- ▶ FLOP: floating point operation
 - ▶ 双精度 (default, 8 bytes) or 单精度 (4 bytes)
 - ▶ 用来衡量任务的工作负载
- ▶ FLOP/s (or FLOPS): floating point operations per second
 - ▶ 用来衡量系统处理目标任务的性能
- ▶ 常用单位
 - ▶ Kilo: $\text{KFLOP/s} = 10^3 \text{ FLOPS}$
 - ▶ Mega: $\text{MFLOP/s} = 10^6 \text{ FLOPS}$
 - ▶ Giga: $\text{GFLOP/s} = 10^9 \text{ FLOPS}$
 - ▶ **Tera: $\text{TFLOP/s} = 10^{12} \text{ FLOPS}$**
 - ▶ Peta: $\text{PFLOP/s} = 10^{15} \text{ FLOPS}$
 - ▶ Exa: $\text{EFLOP/s} = 10^{18} \text{ FLOPS}$
 - ▶ Zetta: $\text{ZFLOP/s} = 10^{21} \text{ FLOPS}$



Nvidia GPU

- Nvidia C870 GPU (1xG80), 2007, 128核, 345.6GFLOPS
- Nvidia K10 GPU (2xGK104), 2012, 3072核, 4.577TFLOPS
- Nvidia P100 GPU(1xGP100), 2016, 3584, 9.3TFLOPS
- Nvidia H100 GPU (1xGH100), 2022, 16896核 (SXM版) , 66.9TFLOPS (单精度) , 989TFLOPS (AI算力, Tensor Core)

GraphCore: Intelligence Processing Unit (IPU)

- 2017, Colossus™ MK1 - GC2 IPU, 1216 Cores, 100-125TFLOPS(fp16)
- 2020, Colossus™ MK2 - GC200 IPU, 1472 Cores, 250-280TFLOPS
- 2023, Colossus™ MK3, ~ 500TFLOPS

FPGA以低功率优势, 但主要用于**AI推理**

Intel CPU

- Xeon Phi 3110X(2012), 第一代, 61核, 1.028TFLOPS
- Xeon Phi 7290F(2016), 最后一代, 72核, 3.456TFLOPS

Google TPU

- TPUv1, 2016, 92TFLOPS(int8, only for inference)
- TPUv2, 2017, 46TFLOPS(bf16)
- TPUv5, 2023, 197TFLOPS(bf16)

国产寒武纪

- 2016, Cambricon-1A(DiaoNaoYu指令集ISA, 获ISCA2016最佳论文奖)
- 2022, [MLU370-X8, 96TFLOPS \(bf16\)](#)

华为昇腾

- 2019, 910A, 256TFLOPS (bf16/f16)
- 2023, 910B, ~ Nvidia A100算力 (312TFLOPS)

	Training		Inference		Generality ⁸⁸	Inference accuracy ⁸⁹
	Efficiency	Speed	Efficiency	Speed		
CPU	1x baseline				Very High	~98-99.7%
GPU	~10-100x	~10-1,000x	~1-10x	~1-100x	High	~98-99.7%
FPGA	-	-	~10-100x	~10-100x	Medium	~95-99%
ASIC	~100-1,000x	~10-1,000x	~100-1,000x	~10-1,000x	Low	~90-98%

Saif M. Khan, AI Chips: What They Are and Why They Matter, 2020

中美AI芯片对比



Type	Firm HQ	Design firm	AI chip	Node (nm)	Fab
GPU	United States	AMD ¹¹⁸	Radeon Instinct	7	TSMC
		Nvidia ¹¹⁹	Tesla V100	12	TSMC
	China	Jingjia Micro ¹²⁰	JM7200	28	Unknown
FPGA	United States	Intel ¹²¹	Agilex	10	Intel
		Xilinx ¹²²	Virtex	16	TSMC
	China	Efinix ¹²³	Trion	40	SMIC
		Gowin Semiconductor ¹²⁴	LittleBee	55	TSMC
		Shenzhen Pango ¹²⁵	Titan	40	Unknown
ASIC	United States	Cerebras ¹²⁶	Wafer Scale Engine	16	TSMC
		Google ¹²⁷	TPU v3	16/12 (est.)	TSMC
		Intel ¹²⁸	Habana	16	TSMC
		Tesla ¹²⁹	FSD computer	10	Samsung
	China	Cambricon ¹³⁰	MLU100	7	TSMC
		Huawei ¹³¹	Ascend 910	7	TSMC
		Horizon Robotics ¹³²	Journey 2	28	TSMC
		Intellifusion ¹³³	NNP200	22	Unknown

Saif M. Khan, AI Chips: What They Are and Why They Matter, 2020



目录 | CONTENT

- 1 人工智能芯片发展介绍
- 2 数据级并行

为什么是多处理机（并行计算机）？



➤ 单处理机有多快？

➤ 考虑一个1 TFLOPS，1 Tbyte的单处理机

- ❑ 数据读取（从内存到CPU）：传输需要一定距离（ r ）
- ❑ 假设我们每个cycle需要得到1个数据，达到1 TFLOPS则需要每秒 10^{12} 次读取，以光速速度传输（ $c=3 \times 10^8$ m/s），则要求： $r < c/10^{12} = 0.3\text{mm}$

➤ 把 1 Tbyte的存储器放在 $0.3\text{mm} \times 0.3\text{mm}$ 的面积上

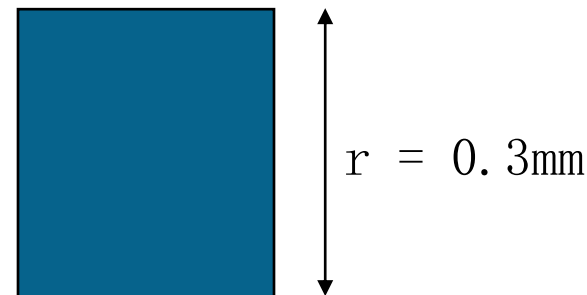
- ❑ 每比特（bit）则需要占用1平方埃（Angstrom），即接近一个原子的大小

➤ 受物理限制，单处理机无法再快了！

- ❑ 为发明更快的处理器，只能选择多处理机

1 TFLOPS,

1 Tbyte 单处理机





并行计算机系统结构的分类

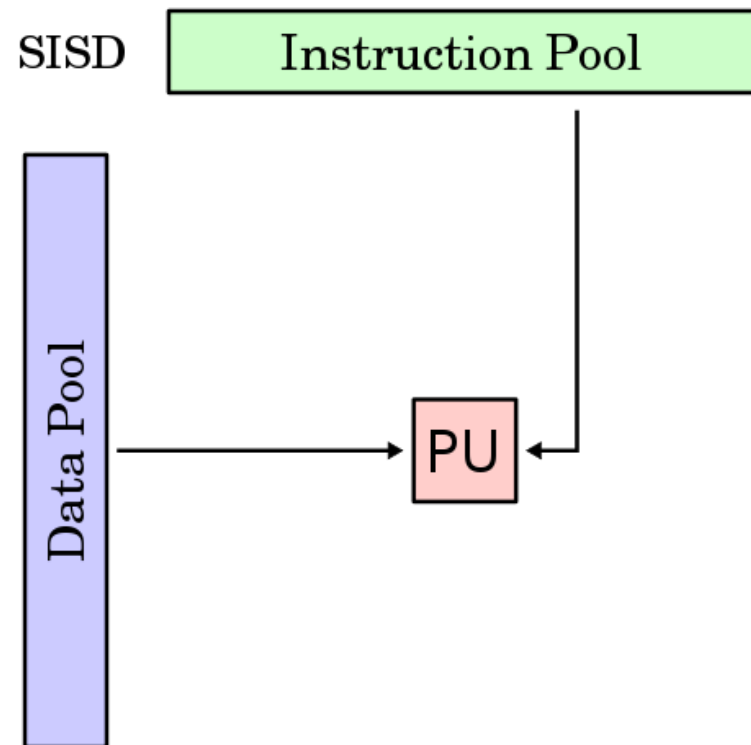
➤ Flynn分类法,1966

- ❑ SISD (Single Instruction Single Data)
- ❑ SIMD (Single Instruction Multiple Data)
- ❑ MISD (Multiple Instruction Single Data) 相对较少
- ❑ MIMD (Multiple Instruction Multiple Data)

SISD (Single Instruction Single Data)



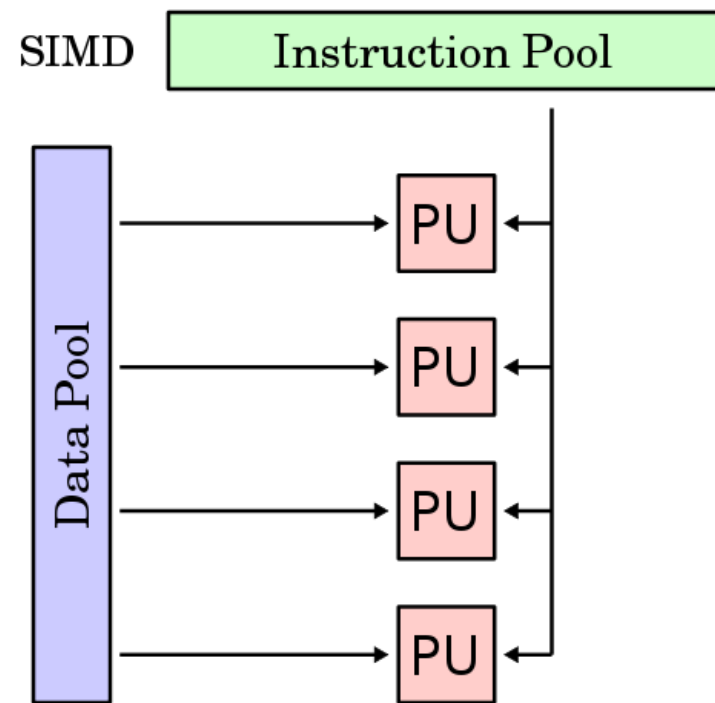
- 单指令流单数据流
- 数据和指令流都是线性执行的
 - ❑ 控制单元 (CU) 从内存中读取单个指令流 (IS)
 - ❑ 随后控制单元生成控制指令给单个处理单元 (PU) 在单个数据流 (DS) 中计算



SIMD (Single Instruction Multiple Data)



- 单指令流多数据流
- 从数据并行角度来实现多处理机的并行技术
 - ❑ 多个数据同时加载进多个处理单元中
 - ❑ 多个数据是用于执行同个指令
- 典型处理器结构
 - ❑ 向量处理器：Intel x86 CPUs: MMX, SSE, and AVX
 - ❑ GPU: Graphics Processing Unit

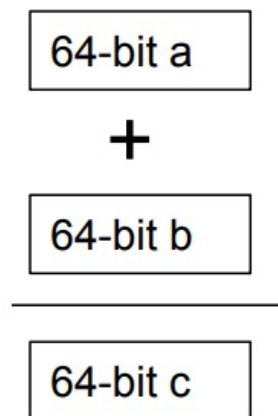


SISD vs. SIMD对比举例



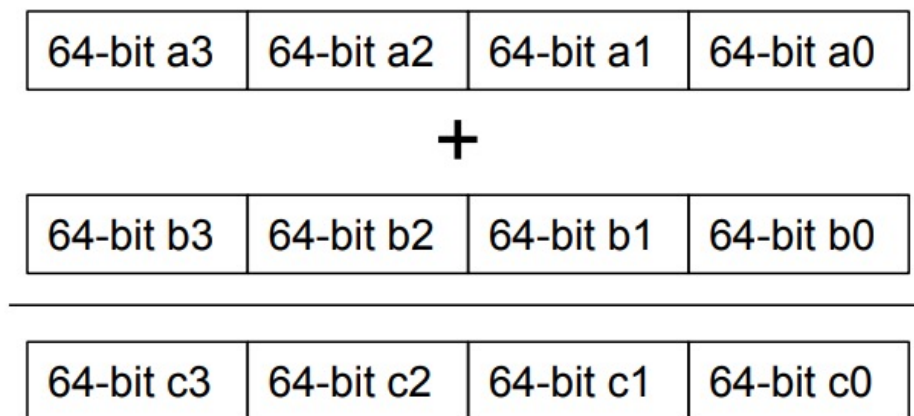
➤ SISD

- ❑ 传统结构
- ❑ 一次操作只产生一个结果



➤ SIMD

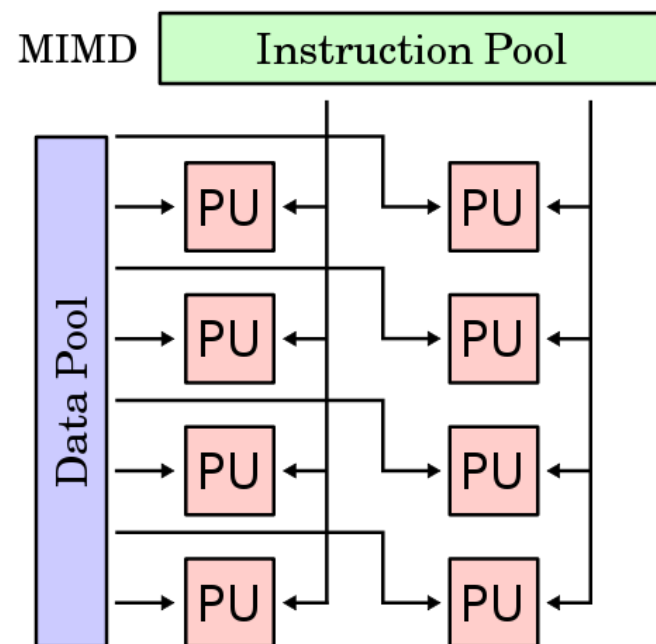
- ❑ 常用主流芯片，如Sandy Bridge: AVX (256 bits)和Cascade Lake: AVX-512 (512 bits)等
- ❑ 一次操作 (256-bit AVX) 产生 $256/64 = 4$ 结果 (double-precision, 64bits)



MIMD (Multiple Instruction Multiple Data)



- 多指令流多数据流
- 支持多个指令同时在多个数据上操作
 - ❑ 独立的芯片或芯片核，每个芯片或核都有自己的控制单元和执行单元
- 目前较为常见的多处理机系统结构
 - ❑ 集中式共享存储器结构
 - ❑ 分布式存储器结构





The most efficient way to execute a vectorizable application is a vector processor.
Jim Smith,
International Symposium on Computer Architecture (1994)



向量体系结构

- **向量**由一组有序、具有相同类型和位数的元素组成。
- 在流水线处理机中，设置向量数据表示和相应的向量指令，称为**向量处理机**。
- 不具有向量数据表示和相应的向量指令的流水线处理机，称为**标量处理机**。
- 典型的向量处理机
 - 1976年 **Cray-1**超级计算机
浮点运算速度达到了**每秒1亿次**
 - CDC Cyber 205 , Cray Y-MP , NEC SX-X/44 , Fujitsu VP2600等
性能达到了**每秒几十亿~几百亿次**浮点运算



向量体系结构

- 基本结构
 - 设置能快速访问的**向量寄存器**，用于存放**源向量、目的向量及中间结果**
 - 结果（分散）写回存储器
 - ✓ 美国的CRAY-1、我国的YH-1巨型机
- 寄存器由编译器控制
 - 隐藏访存延迟
 - 充分利用存储器带宽



向量体系结构

- 以CRAY-1机为例
 - 美国CRAY公司
 - 1976年
 - 每秒1亿次浮点运算
 - 时钟周期：12.5ns

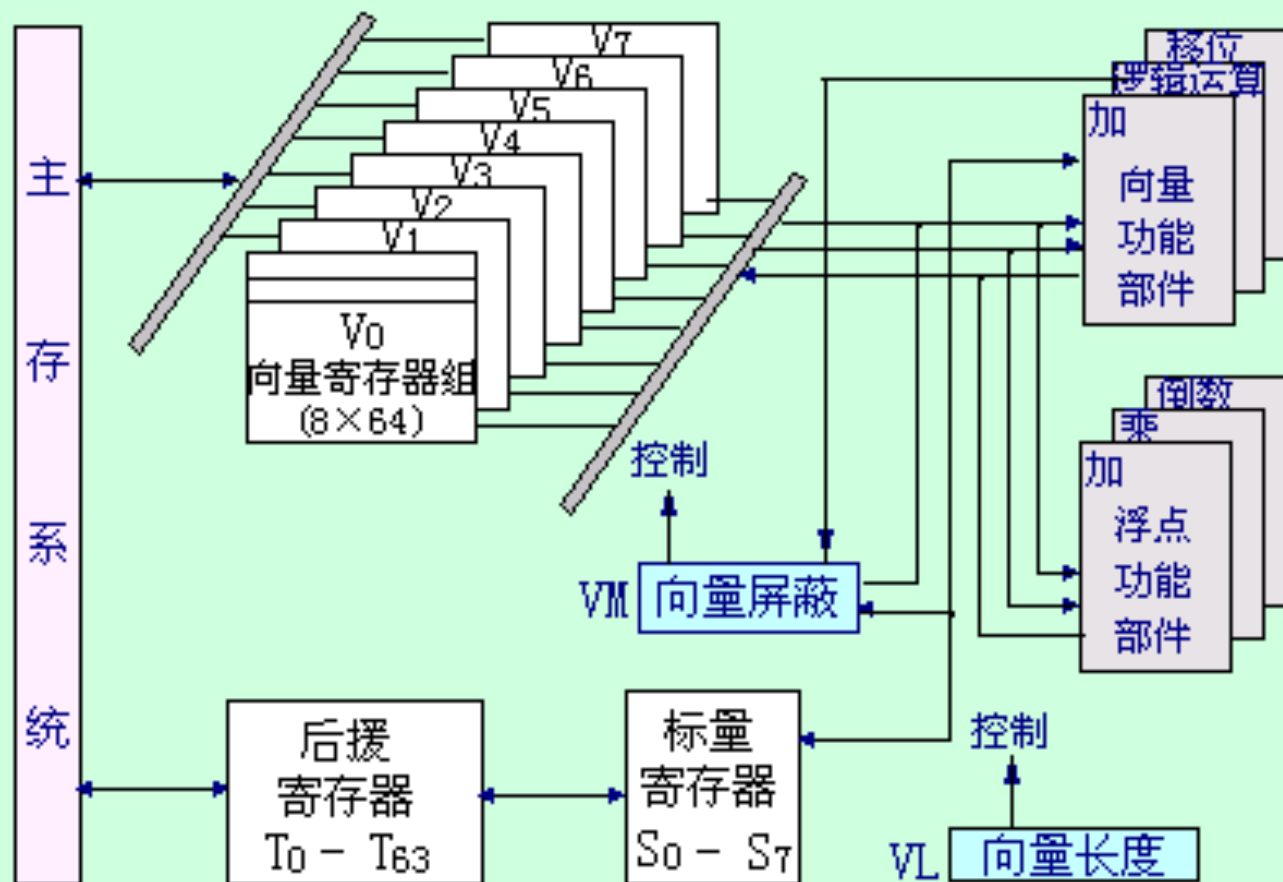
CRAY-1的基本结构

- 功能部件
 - 共有12条可并行工作的单功能流水线，可分别流水地进行地址、向量、标量的各种运算。

CRAY-1机



CRAY-1的基本结构



➤ 6个单功能流水部件：进行向量运算

- 整数加（3拍）
- 逻辑运算（2拍）
- 移位（4拍）
- 浮点加（6拍）
- 浮点乘（7拍）
- 浮点迭代求倒数（14拍）

括号中的数字为其流水经过的时间，每拍为一个时钟周期，即12.5ns。

➤ 向量寄存组V

- 由512个64位的寄存器组成，分成8块。
- 编号： $V_0 \sim V_7$
- 每一个块称为一个向量寄存器，可存放一个长度（即元素个数）不超过64的向量。
- 每个向量寄存器可以每拍向功能部件提供一个数据元素，或者每拍接收一个从功能部件来的结果元素。

➤ 向量屏蔽寄存器VM

- 64位，每一位对应于向量寄存器的一个单元。
- 作用：用于向量的归并、压缩、还原和测试操作、对向量某些元素的单独运算等。
- 标量寄存器S和快速暂存器T
 - ✓ 标量寄存器有8个： $S_0 \sim S_7$ 64位
 - ✓ 快速暂存器T用于在标量寄存器和存储器之间提供缓冲。

➤ CRAY-1向量处理的一个显著特点

- ❑ 每个向量寄存器 V_i 都有连到6个向量功能部件的单独总线。
- ❑ 每个向量功能部件也都有把运算结果送回向量寄存器组的总线。

➤只要不出现 **V_i 冲突**和**功能部件冲突**，各 V_i 之间和各功能部件之间都能并行工作，大大加快了向量指令的处理。

□ **V_i 冲突**：并行工作的各向量指令的源向量或结果向量使用了相同的 V_i 。

例如：源向量相同

$$V_3 \leftarrow V_1 + V_2$$

$$V_5 \leftarrow V_4 \wedge V_1$$

□**功能部件冲突**：并行工作的各向量指令要使用同一个功能部件。

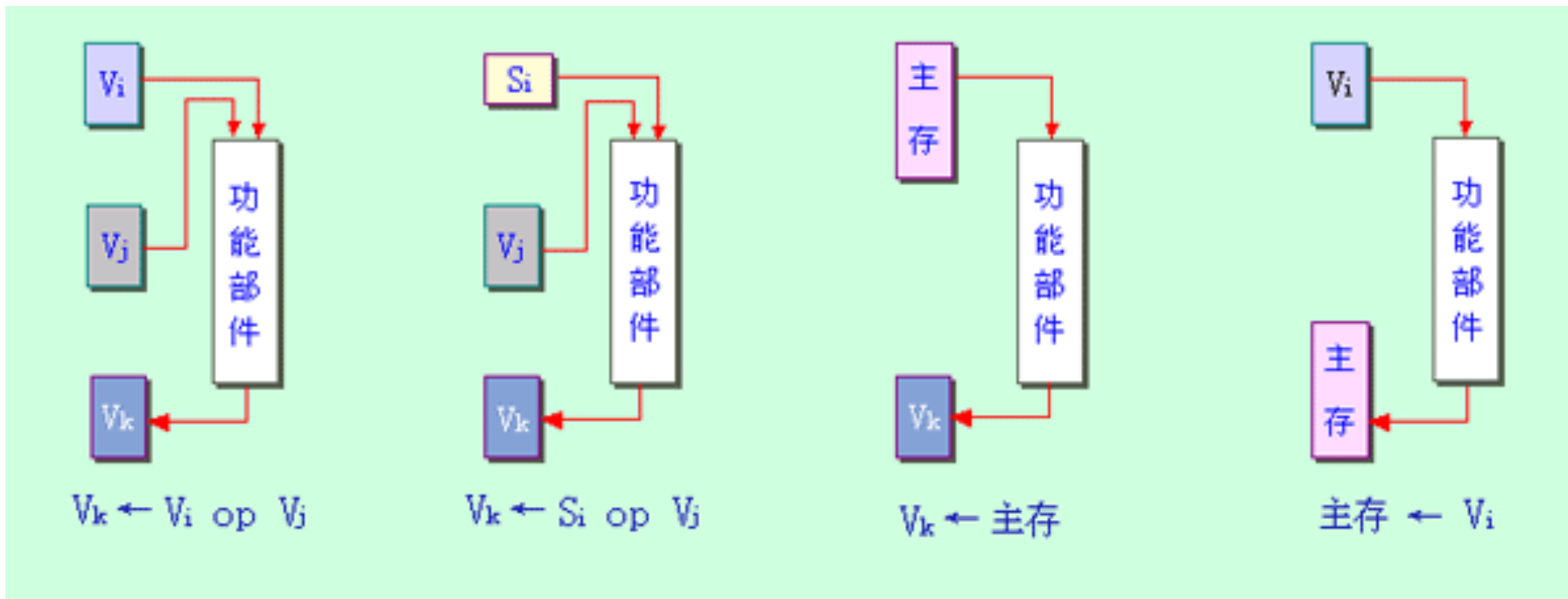
例如：都需使用乘法功能部件

$$V_3 \leftarrow V_1 \times V_2$$

$$V_5 \leftarrow V_4 \times V_6$$

CRAY-1向量指令类型

- $V_k \leftarrow V_i \text{ op } V_j$
- $V_k \leftarrow S_i \text{ op } V_j$
- $V_k \leftarrow \text{主存}$
- $\text{主存} \leftarrow V_i$

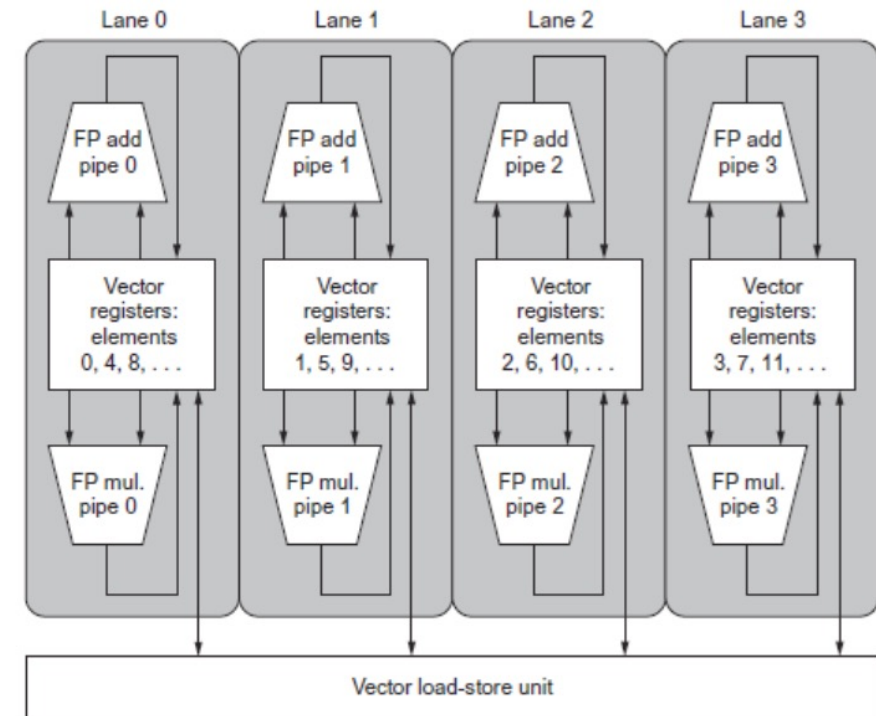
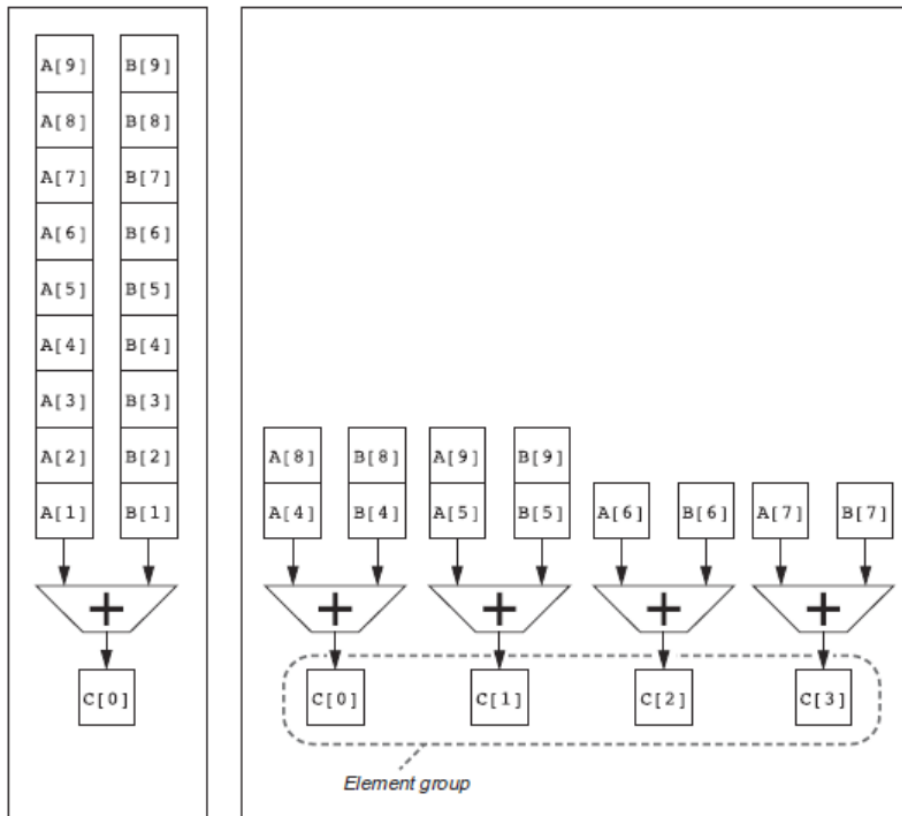


- Start up time 启动开销
 - 功能单元延时较高
 - 例如 Cray-1机中
 - 浮点加法 => 6 clock cycles
 - 浮点乘法 => 7 clock cycles
 - 浮点除法 => 20 clock cycles
 - Vector load => 12 clock cycles
- 性能提升:
 - 每个时钟周期处理多于1个元素
 - 处理非Non-64位宽的向量
 - 向量代码中存在分支语句
 - 内存系统优化
 - 多维矩阵
 - 稀疏矩阵
 - 向量处理机的编程

Multiple Lanes



- Element n of vector register A is “hardwired” to element n of vector register B
 - Allows for multiple hardware lanes



Vector Length Register 向量长度寄存器



```
for (i=0; i < n; i=i+1) Y[i] = a * X[i] + Y[i];
```

```
    vsetdcfg 2 DP FP    # Enable 2 64b Fl.Pt. registers
    fld f0,a            # Load scalar a
loop: setvl t0,a0        # vl = t0 = min(mvl,n)
    vld v0,x5           # Load vector X
    slli t1,t0,3        # t1 = vl * 8 (in bytes)
    add x5,x5,t1        # Increment pointer to X by vl*8
    vmul v0,v0,f0       # Vector-scalar mult
    vld v1,x6           # Load vector Y
    vadd v1,v0,v1       # Vector-vector add
    sub a0,a0,t0        # n -= vl (t0)
    vst v1,x6           # Store the sum into Y
    add x6,x6,t1        # Increment pointer to Y by vl*8
    bnez a0,loop        # Repeat if n != 0
    vdisable            # Disable vector regs
```

Vector Mask Registers 向量屏蔽寄存器



- Consider:

```
for (i = 0; i < 64; i=i+1)
    if (X[i] != 0)
        X[i] = X[i] - Y[i];
```

Use predicate register to “disable” elements:

vsetdcfg	2*FP64	# Enable 2 64b FP vector regs
vsetpcfgi	1	# Enable 1 predicate register
vld	v0,x5	# Load vector X into v0
vld	v1,x6	# Load vector Y into v1
fmv.d.x	f0,x0	# Put (FP) zero into f0
vpne	p0,v0,f0	# Set p0(i) to 1 if v0(i)!=f0
vsub	v0,v0,v1	# Subtract under vector mask
vst	v0,x5	# Store the result in X
vdisable		# Disable vector registers
vpdisable		# Disable predicate registers

Memory Banks



- Memory system must be designed to support high bandwidth for vector loads and stores
- Spread accesses across multiple banks
 - Control bank addresses independently
 - Load or store non sequential words (need independent bank addressing)
 - Support multiple vector processors sharing the same memory
- Example:
 - 32 processors, each generating 4 loads and 2 stores/cycle
 - Processor cycle time is 2.167 ns, SRAM cycle time is 15 ns
 - How many memory banks needed?
 - $32 \times (4+2) \times 15 / 2.167 = \sim 1330$ banks

- Consider:

```
for (i = 0; i < 100; i=i+1)
```

```
  for (j = 0; j < 100; j=j+1) {
```

```
    A[i][j] = 0.0;
```

```
    for (k = 0; k < 100; k=k+1)
```

```
      A[i][j] = A[i][j] + B[i][k] * D[k][j];
```

```
  }
```

- Must vectorize multiplication of rows of B with columns of D
- Use *non-unit stride*
- Bank conflict (stall) occurs when the same bank is hit faster than bank busy time:
 - $\#banks / LCM(stride, \#banks) < \text{bank busy time}$

Scatter-Gather



- Consider:

for (i = 0; i < n; i=i+1)

$A[K[i]] = A[K[i]] + C[M[i]];$

- Use index vector:

vsetdcfg	4*FP64	# 4 64b FP vector registers
vld	v0, x7	# Load K[]
vldx	v1, x5, v0	# Load A[K[]]
vld	v2, x28	# Load M[]
vldi	v3, x6, v2	# Load C[M[]]
vadd	v1, v1, v3	# Add them
vstx	v1, x5, v0	# Store A[K[]]
vdisable		# Disable vector registers

向量处理机的编程



- Compilers can provide feedback to programmers
- Programmers can provide hints to compiler

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15



- Implementations:
 - Intel MMX (1996)
 - Eight 8-bit integer ops or four 16-bit integer ops
 - Streaming SIMD Extensions (SSE) (1999)
 - Eight 16-bit integer ops
 - Four 32-bit integer/fp ops or two 64-bit integer/fp ops
 - Advanced Vector Extensions (2010)
 - Four 64-bit integer/fp ops
 - AVX-512 (2017)
 - Eight 64-bit integer/fp ops

操作数必须是连续的且内存对齐的

Example SIMD Code



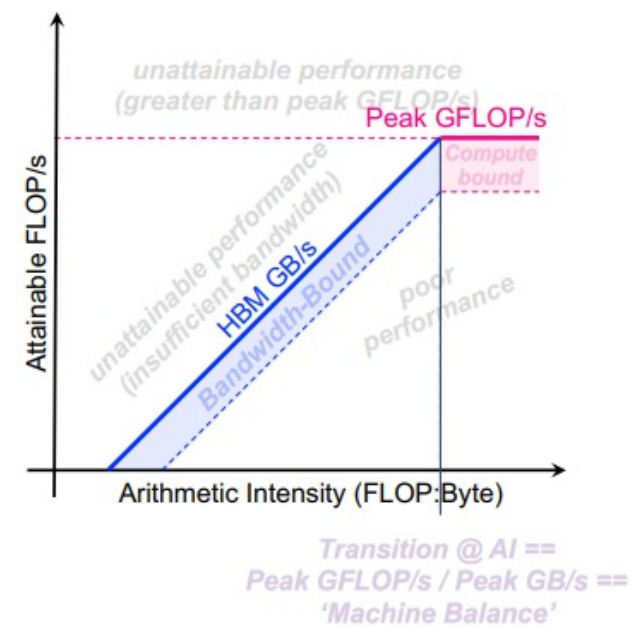
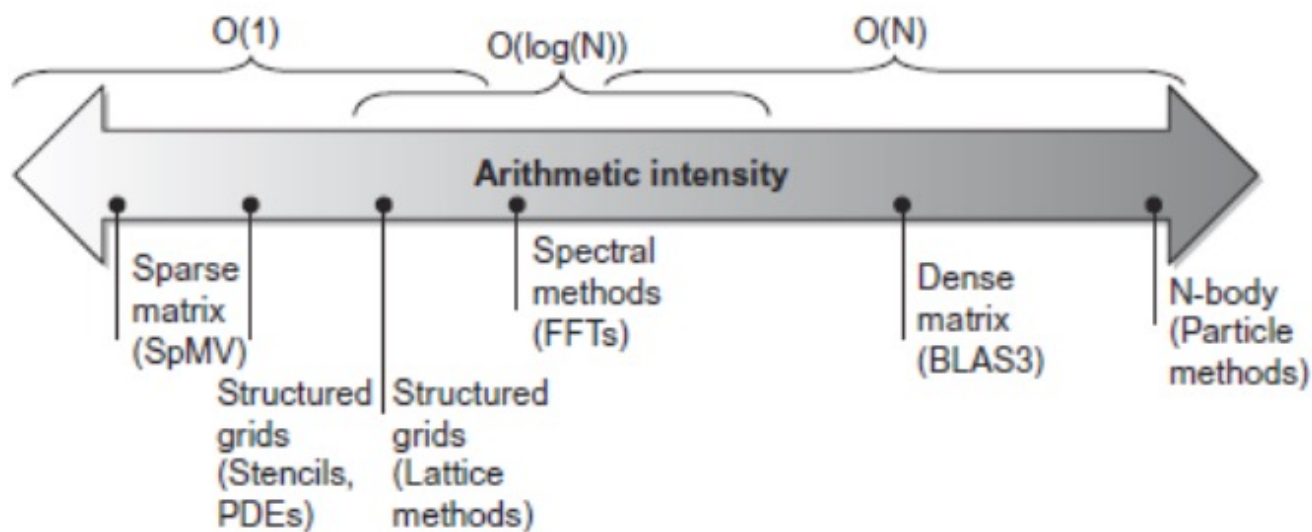
- Example DXPY:

fld	f0,a	# Load scalar a
splat.4D	f0,f0	# Make 4 copies of a
addi	x28,x5,#256	# Last address to load
Loop: fld.4D	f1,0(x5)	# Load X[i] ... X[i+3]
fmul.4D	f1,f1,f0	# a x X[i] ... a x X[i+3]
fld.4D	f2,0(x6)	# Load Y[i] ... Y[i+3]
fadd.4D	f2,f2,f1	# a x X[i]+Y[i]...
		# a x X[i+3]+Y[i+3]
fsd.4D	f2,0(x6)	# Store Y[i]... Y[i+3]
addi	x5,x5,#32	# Increment index to X
addi	x6,x6,#32	# Increment index to Y
bne	x28,x5,Loop	# Check if done

Roofline 性能模型



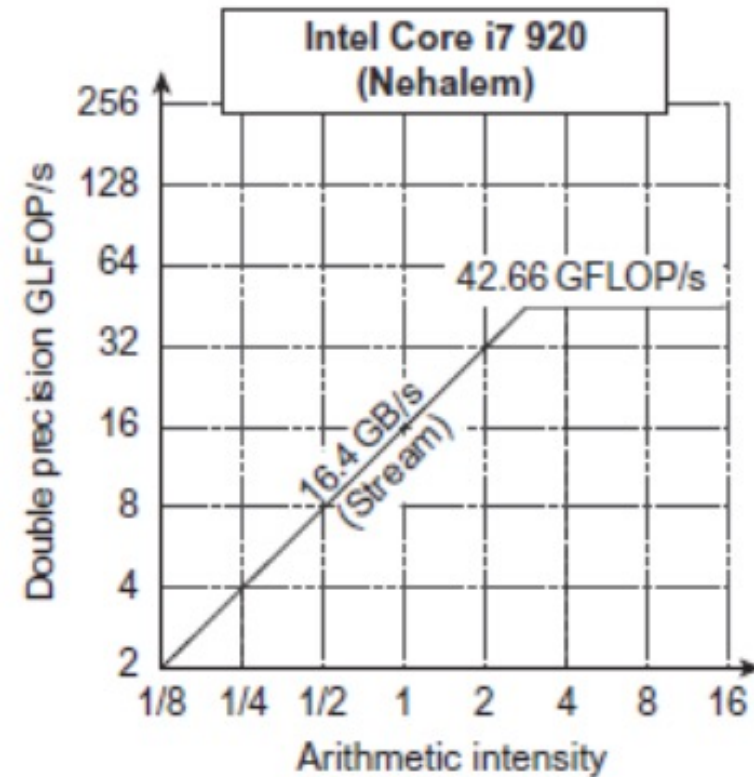
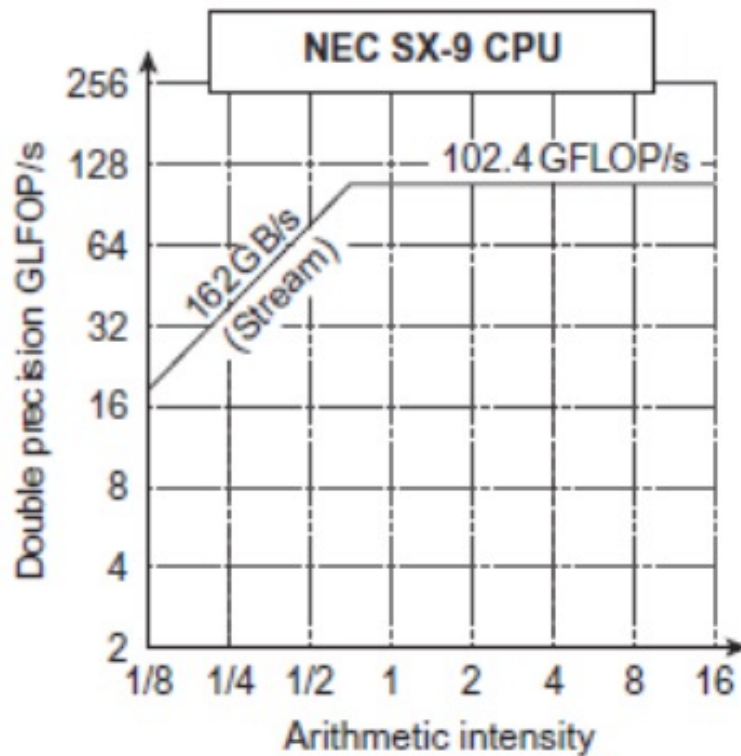
- Basic idea:
 - Plot peak floating-point throughput as a function of arithmetic intensity
 - Ties together floating-point performance and memory performance for a target machine
- Arithmetic intensity
 - Floating-point operations per byte read



Examples



- Attainable GFLOPs/sec = (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Perf.)



Graphical Processing Units (GPU)



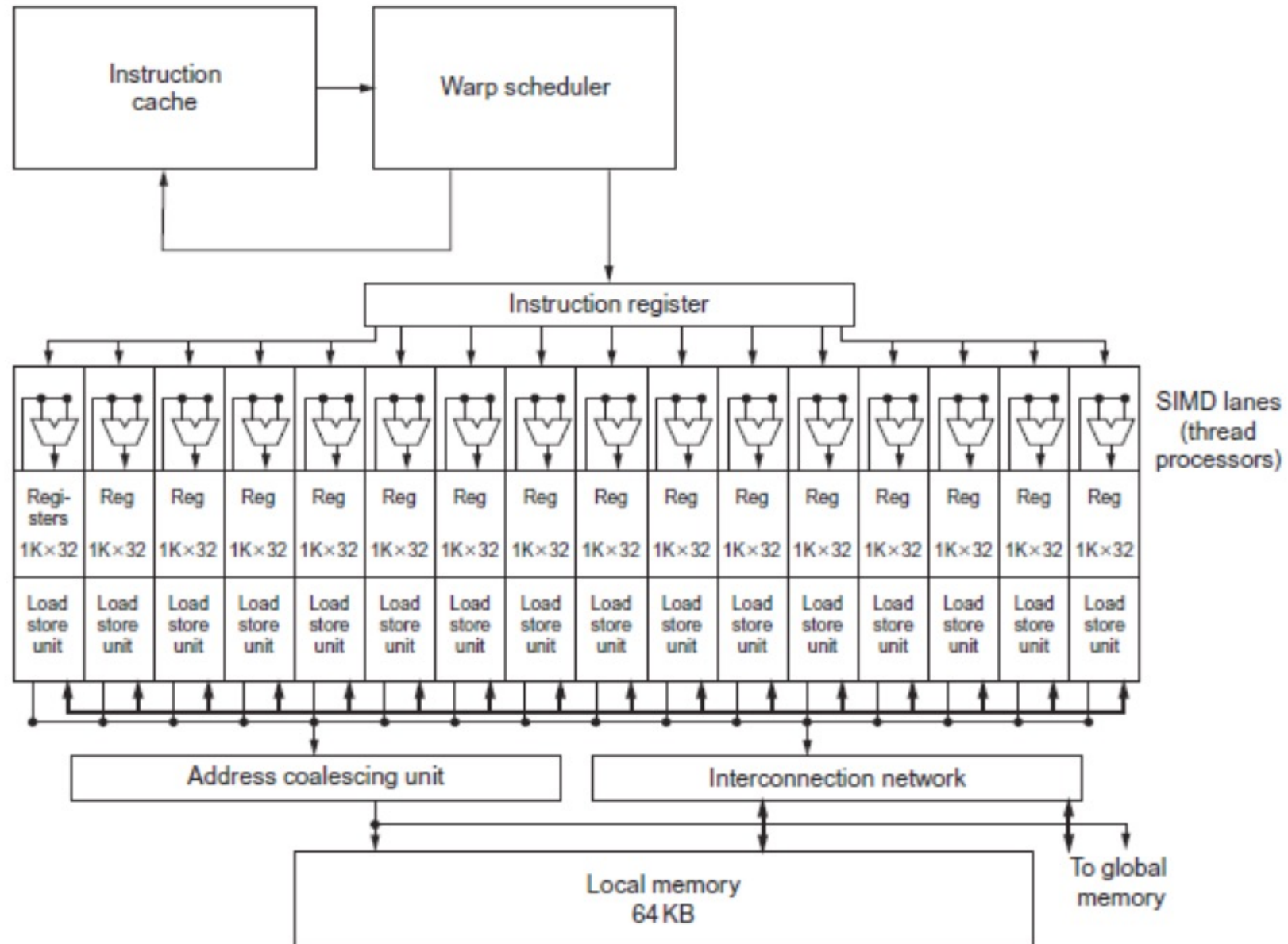
- Basic idea:
 - Heterogeneous execution model
 - CPU is the *host*, GPU is the *device*
 - Develop a C-like programming language for GPU
 - Unify all forms of GPU parallelism as *CUDA thread*
 - Programming model is “Single Instruction Multiple Thread”

GPU与向量处理机的差异



- Similarities to vector machines:
 - Works well with data-level parallel problems
 - Scatter-gather transfers
 - Mask registers
 - Large register files
- Differences:
 - No scalar processor
 - Uses multithreading to hide memory latency
 - Has many functional units, as opposed to a few deeply pipelined units like a vector processor

GPU Organization



NVIDIA Instruction Set Architecture



- ISA is an abstraction of the hardware instruction set
 - “Parallel Thread Execution (PTX)”
 - opcode.type d,a,b,c;
 - Uses virtual registers
 - Translation to machine code is performed in software
 - Example:

```
shl.s32      R8, blockIdx, 9    ; Thread Block ID * Block size (512 or 29)
add.s32      R8, R8, threadIdx ; R8 = i = my CUDA thread ID
ld.global.f64 RD0, [X+R8]      ; RD0 = X[i]
ld.global.f64 RD2, [Y+R8]      ; RD2 = Y[i]
mul.f64      R0D, RD0, RD4      ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64      R0D, RD0, RD2      ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0       ; Y[i] = sum (X[i]*a + Y[i])
```


GV100

- 每个SM (计算单元)
 - FP32单元 : 64
 - FP64单元 : 32
 - int32单元 : 64
 - Tensor Core单元 : 8
- Warp调度器 : 4
- 加载/存储单元 : 32
- SFU : 16
- 每个SM (存储单元)
 - 寄存器 : 256KB
 - L1高速缓存/共享内存 : 128KB

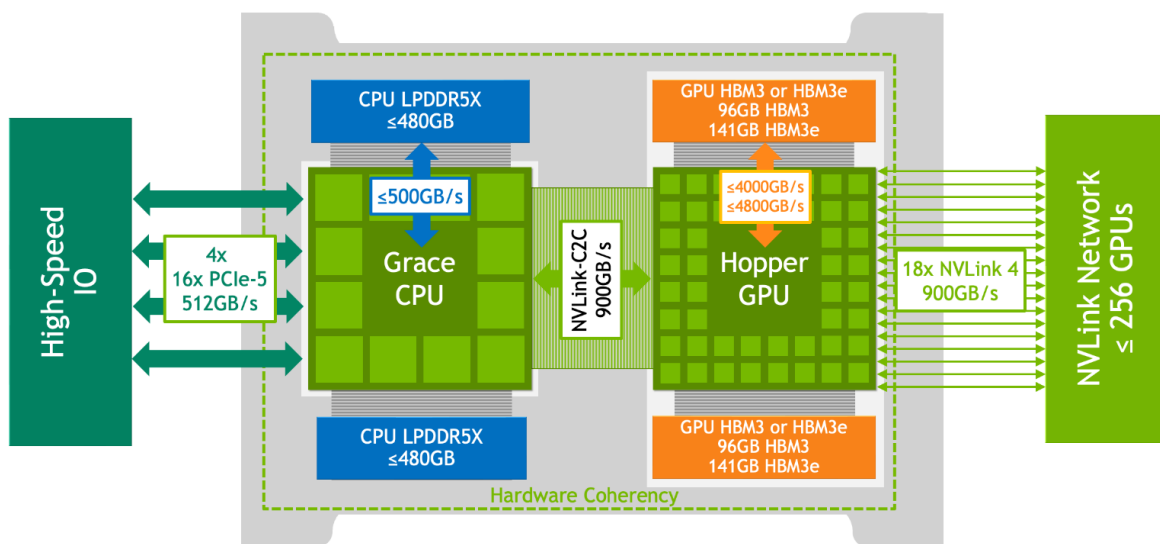


NVIDIA DGX GH200

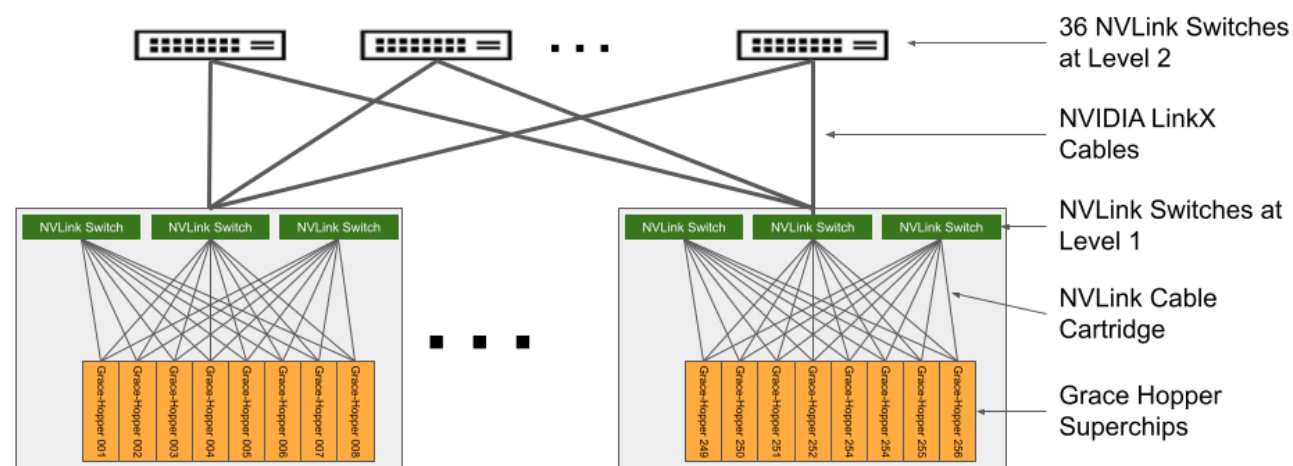


- GH200 Grace Hopper Superchip
- DGX GH200 Supercomputer

Feature	Description
Grace CPU cores (number)	Up to 72 cores
CPU LPDDR5X bandwidth (GB/s)	Up to 500GB/s
GPU HBM bandwidth (GB/s)	4TB/s HBM3 4.9TB/s HBM3e
NVLink-C2C bandwidth (GB/s)	900GB/s total, 450GB/s per direction
CPU LPDDR5X capacity (GB)	Up to 480GB
GPU HBM capacity (GB)	96GB HBM3 144GB HBM3e
PCIe Gen 5 Lanes	64x



Fully Connected NVLink across 256 GPUs



Tensor Processing Unit (TPU)

张量处理单元 (英文 : Tensor Processing Unit , 简称 : TPU) , 是 Google 开发的专用集成电路 (ASIC)

- The Matrix Unit: 65,536 (256x256) 8-bit multiply-accumulate units
- 700 MHz clock rate
- Peak: 92T operations/second
- $65,536 * 2 * 700M$
- >25X as many MACs vs GPU
- >100X as many MACs vs CPU
- 4 MiB of on-chip Accumulator memory
- 24 MiB of on-chip Unified Buffer (activation memory)
- 3.5X as much on-chip memory vs GPU
- Two 2133MHz DDR3 DRAM channels
- 8 GiB of off-chip weight DRAM memory

65536个8位乘累加运算单元 (MAC , 乘积累加单元)

峰值 : 每秒92T(tera,万亿)次操作

拥有的MAC数量是GPU的25倍

拥有的MAC数量是CPU的100倍

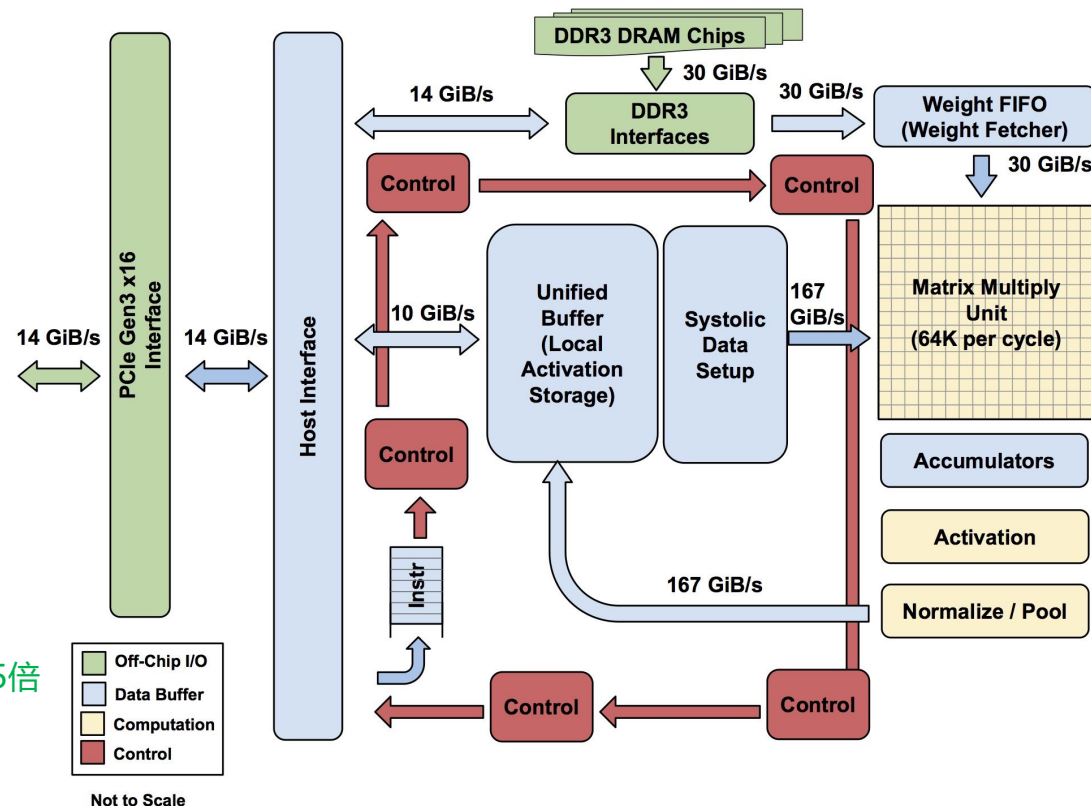
4 MiB的片上累加器内存

24 MiB的片上统一缓冲区

相对于GPU , 片上内存容量是GPU的3.5倍

两个2133MHz的DDR3 DRAM通道

8 GiB的片外权重DRAM内存



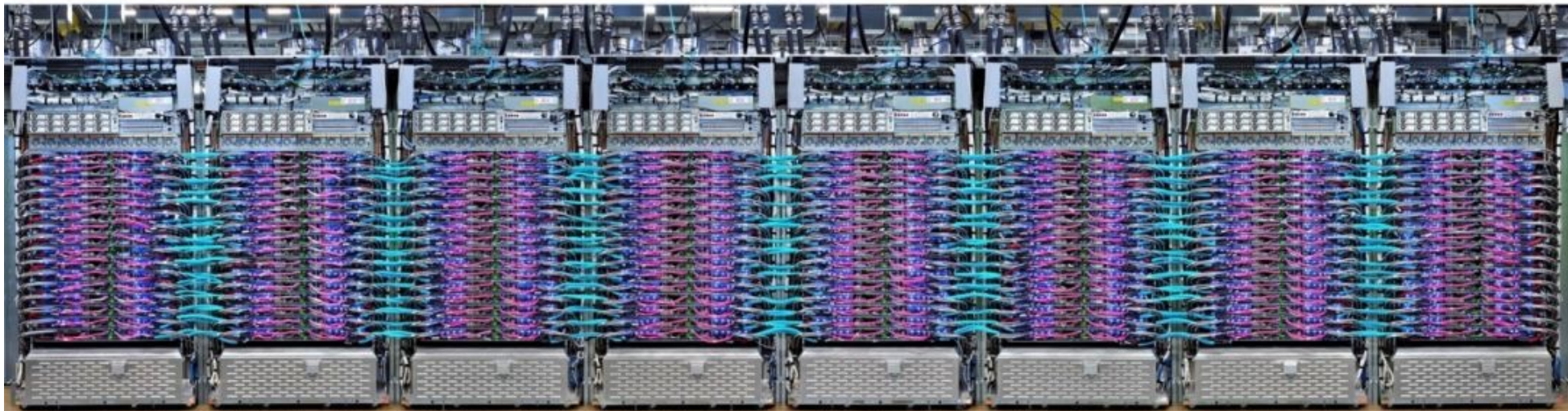
TPU架构设计

Google TPUv2



Google TPUv2 Pod (2017)
11.5 petaflops
4 TB HBM
2-D toroidal mesh network
Training and inference

Google TPUv3 Pod (2018)
>100 petaflops

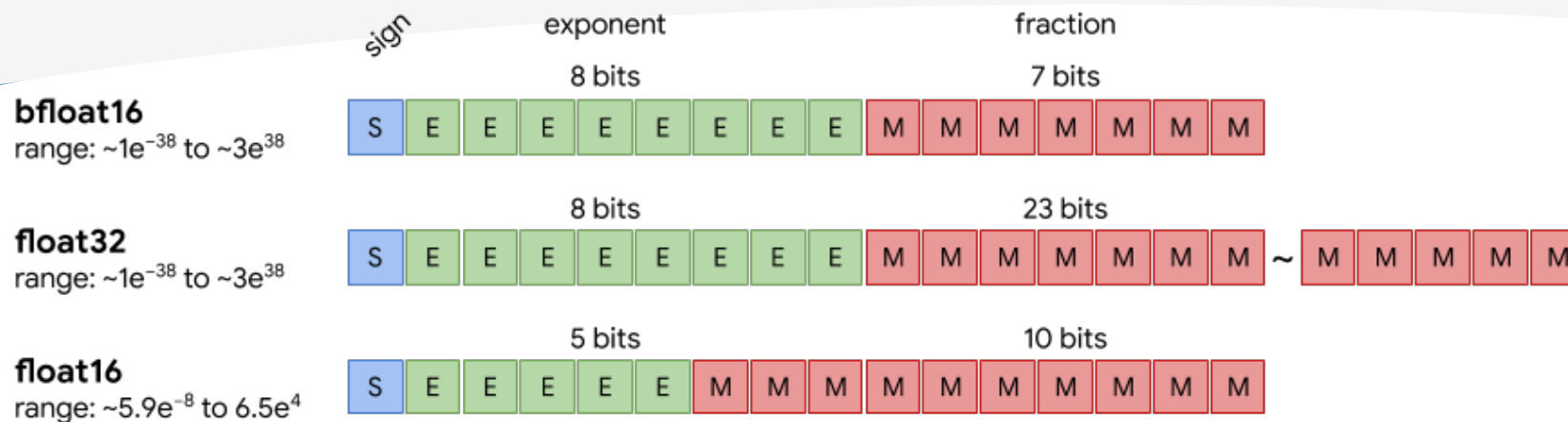


不同版本TPU的演化



Chip	TPUv1	TPUv2	TPUv3
Announced	2016	May-17	May-18
Access	Internal-Only	Service Beta	<i>Undisclosed</i>
Introduction	2015	Feb 2018	<i>Undisclosed</i>
Process	28nm	<i>20nm est.</i>	<i>16/12nm est.</i>
Die Size	~300mm ²	<i>Undisclosed</i>	<i>Undisclosed</i>
TOPS	92 / 23	45	90
Matrix Input	INT8 / INT16	bfloat16	bfloat16
Memory	8GB DDR3	16GB HBM	32GB HBM
CPU Interface	PCIe 3.0 x16	PCIe 3.0 x8	<i>PCIe 3.0 x8 est.</i>
Power Consumption	40W	<i>200-250W est.</i>	<i>200W est.</i>

bfloat (brain floating point)



A Brief Guide to Floating Point Formats

fp32: Single-precision IEEE Floating Point Format

Range: $\sim 1e^{-38}$ to $\sim 3e^{38}$



fp16: Half-precision IEEE Floating Point Format

Range: $\sim 5.96e^{-8}$ to 65504



bfloat16: Brain Floating Point Format

Range: $\sim 1e^{-38}$ to $\sim 3e^{38}$





授课教师：王强

邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You